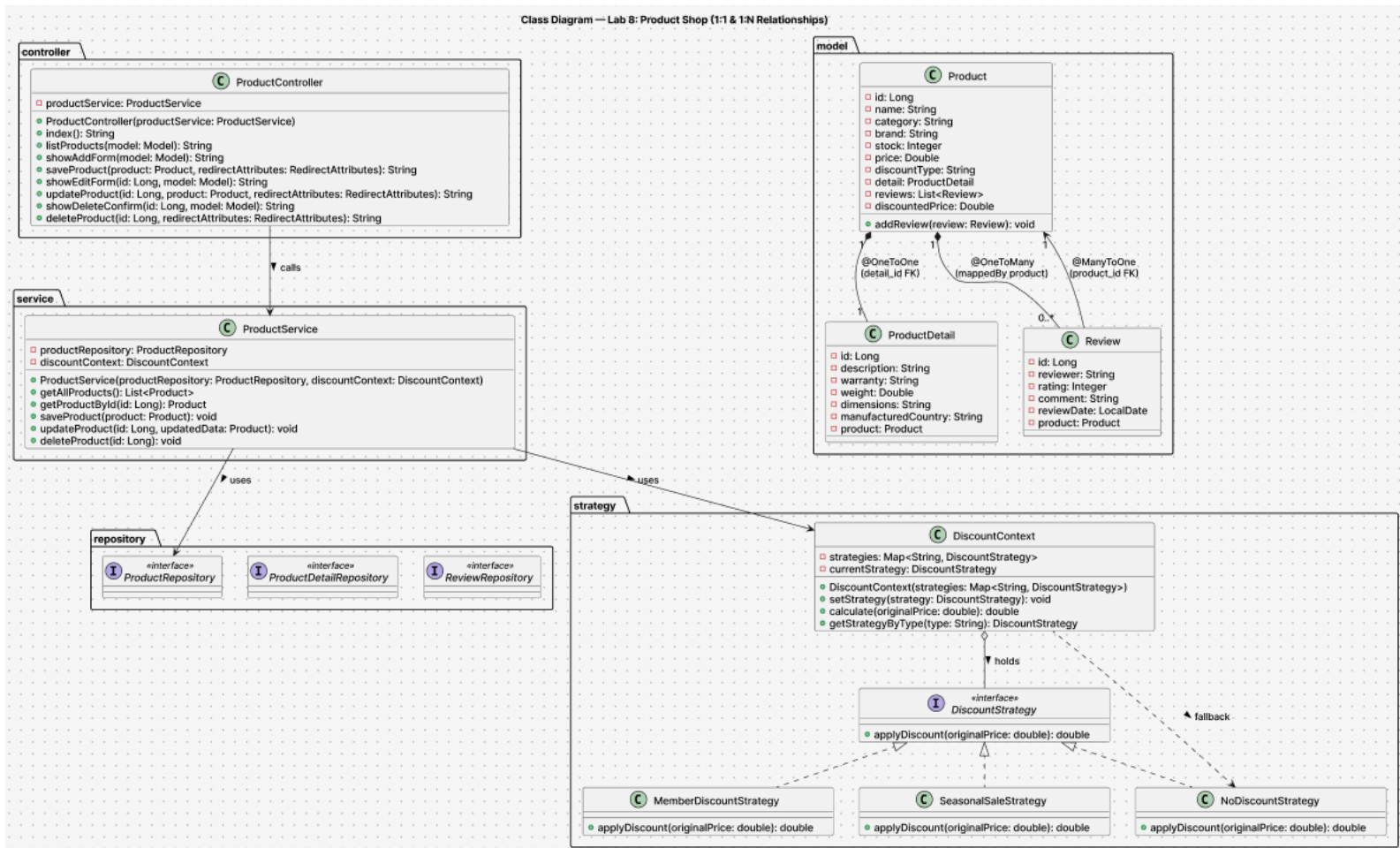


ภาคที่ 0: ภาพรวมสถาปัตยกรรมระบบและแผนภาพคลาส (Class Diagram & Architecture Overview)

แผนภาพ Class Diagram นี้แสดงถึงโครงสร้างเชิงสถาปัตยกรรมของระบบ Product Shop ที่ได้รับการออกแบบตามรูปแบบ Layered Architecture ร่วมกับ Domain Model (JPA Entities) และ Strategy Design Pattern โดยมีการแบ่งแยกคลาสออกเป็น 5 Packages เพื่อให้ระบบมีความเป็นอิสระต่อกัน (Loose Coupling) และง่ายต่อการทดสอบบำรุงรักษา



1. controller (Presentation Layer): `ProductController` ทำหน้าที่เป็นจุดรับคำขอ (HTTP Request) จากเว็บเบราว์เซอร์ และส่งคืนหน้า View (Thymeleaf)
2. service (Business Logic Layer): `ProductService` มี Dependency สองตัวคือ `productRepository`: ใช้สำหรับบันทึกและสืบค้นข้อมูลลงฐานข้อมูล PostgreSQL และ `discountContext` ใช้สำหรับคำนวณราคาสุทธิหลังหักส่วนลดของสินค้าแต่ละชิ้นก่อนส่งไปแสดงผล
3. repository (Data Access Layer): `ProductRepository`, `ProductDetailRepository`, `ReviewRepository` ออกแบบตามหลัก ISP (Interface Segregation Principle) โดยแยกหน้าที่ของแต่ละ Entity ออกจากกัน ไม่รวมเป็น Interface ขนาดใหญ่เพียงตัวเดียว

4. model (Domain Entity Layer): `Product`: เป็น Entity หลักถือ Attribute พื้นฐาน เช่น `name`, `price`, `stock`, `discountType` และมีฟิลด์ชั่วคราว `@Transient` `discountedPrice`
`ProductDetail`: Entity ข้อมูลเชิงลึกของสินค้า เช่น `warranty`, `dimensions`, `weight`
`Review`: Entity สำหรับเก็บความคิดเห็นและคะแนนดาวของผู้ใช้งาน
5. strategy (Strategy Design Pattern Layer): `DiscountStrategy`, `DiscountContext` และ Concrete Strategies: คลาสที่นำ Interface ไป Implement เป็นสูตรเฉพาะตัว มี `NoDiscountStrategy`, `MemberDiscountStrategy`, `SeasonalSaleStrategy`

ภาคที่ 1: หลักการออกแบบซอฟต์แวร์ (Software Design Principles)

1.1 การประยุกต์ใช้ SOLID Principles กับการออกแบบระบบและ Entity

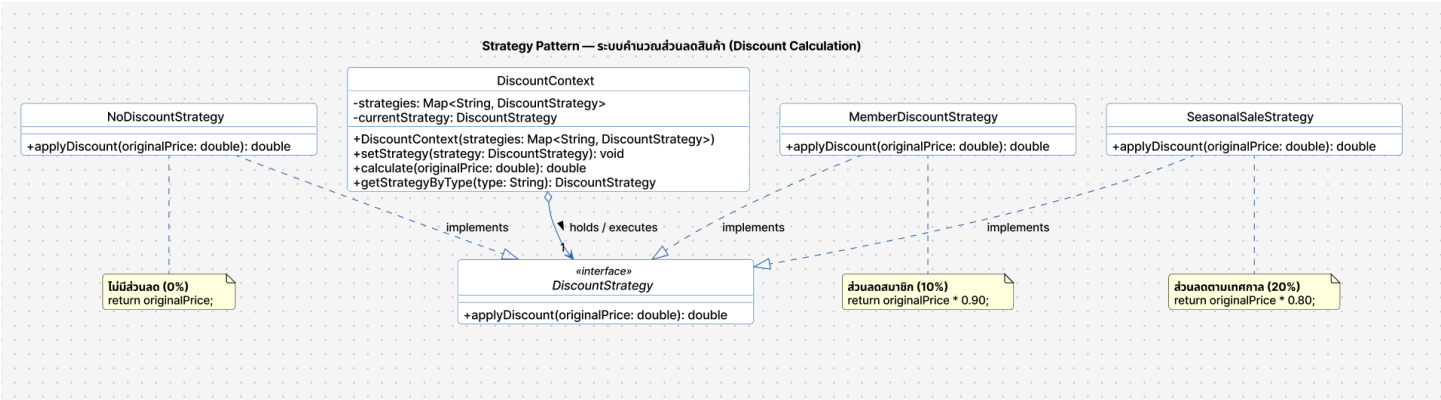
ตัว ย่อ	หลักการ (Principle)	การประยุกต์ใช้กับ Entity และสถาปัตยกรรมของโปรเจกต์
S	Single Responsibility Principle (SRP)	ระดับ Entity: แยกข้อมูลเสริมที่ไม่จำเป็นต้อง Query บ่อยๆ ออกเป็น ProductDetail และแยกข้อมูลรีวิวที่เพิ่มได้ไม่จำกัดเป็น Review ไม่รวมรวมทุก Attribute ไว้ในตาราง Product ตารางเดียว ระดับ Layered Architecture: แบ่งหน้าที่ตามเลเยอร์อย่างชัดเจน ได้แก่ ProductController (รับ-ส่ง HTTP เท่านั้น) ProductService (ประมวลผล Business Logic เท่านั้น), Repository (จัดการการเข้าถึงฐานข้อมูลเท่านั้น) และ DiscountContext (จัดการการคำนวณส่วนลดเท่านั้น)
O	Open/Closed Principle (OCP)	ระดับ Entity/Database: สามารถเพิ่มตาราง Review เพื่อเก็บประวัติความคิดเห็นได้โดยไม่ต้องแก้ไขโครงสร้างตาราง products ระดับ Strategy Pattern: ระบบคำนวณส่วนลดใช้ Spring Component Map Injection ใน DiscountContext ทำให้สามารถเพิ่มคลาสส่วนลดใหม่ (เช่น VIPDiscountStrategy) ได้ทันทีโดยไม่ต้องกลับมาแก้ไขโค้ดเดิมใน DiscountContext เลยแม้แต่บรรทัดเดียว
L	Liskov Substitution Principle (LSP)	คลาสกลยุทธ์ส่วนลดทุกตัว (NoDiscountStrategy, MemberDiscountStrategy, SeasonalSaleStrategy) สามารถนำไปใช้แทน Interface DiscountStrategy ได้อย่างไร้รอยต่อ โดยไม่ทำให้ระบบทำงานผิดพลาด อินเทอร์เฟซ ProductRepository, ProductDetailRepository และ ReviewRepository สามารถใช้ทดแทน JpaRepository พื้นฐานของ Spring Data ได้อย่างสมบูรณ์
I	Interface Segregation Principle (ISP)	อินเทอร์เฟซ DiscountStrategy มีเฉพาะเมธอด applyDiscount(double originalPrice) ที่จำเป็นต่อการคำนวณ ไม่ยืดเยืดยาวเกินไปที่เกี่ยวข้อง แยก Repository ออกเป็น 3 อินเทอร์เฟซตามแต่ละ Entity อย่างอิสระ ไม่รวมเป็น Big Repository ขนาดใหญ่เพียงตัวเดียว
D	Dependency Inversion Principle (DIP)	ทุกโมดูลระดับสูงขึ้นอยู่กับ Abstraction: ProductController เรียกใช้ ProductService ผ่าน Constructor Injection และ Service เชื่อมต่อกับฐานข้อมูลผ่าน Repository Interface (ไม่ได้ขึ้นกับ Implementation จริง) DiscountContext รับ Dependencies ของกลยุทธ์ส่วนลดผ่าน Map<String, DiscountStrategy> ที่ Spring IoC Container ฉีดเข้ามาให้ แทนการใช้คำสั่ง new เรียกหา Concrete Class

1.2 ความแตกต่างของ One-to-One (1:1) และ One-to-Many (1:N)

หัวข้อ	One-to-One (1:1)	One-to-Many (1:N)
นิยามความสัมพันธ์	Product (1) ————— (1) ProductDetailสินค้า 1 ชิ้น มีรายละเอียดเสริมได้เพียง 1 รายการ	Product (1) ————— (N) Reviewสินค้า 1 ชิ้น มีรีวิวได้ไม่จำกัดจำนวน (0 ถึง N)
ตำแหน่งการวาง Foreign Key (FK)	ฝั่งที่เป็น Owner Side โดยในระบบนี้คือตาราง products จะถือคอลัมน์ detail_id	ฝั่ง Many Side เสมอ โดยในระบบนี้คือตาราง reviews จะถือคอลัมน์ product_id
Annotations ที่ใช้งาน	ฝั่ง Owner: @OneToOne + @JoinColumn(name = "detail_id") ฝั่ง Inverse: @OneToOne(mappedBy = "detail")	ฝั่ง One: @OneToMany(mappedBy = "product") ฝั่ง Many: @ManyToOne + @JoinColumn(name = "product_id")
กรณีที่เหมาะสมในการนำไปใช้	ข้อมูลเสริมหรือข้อมูลเฉพาะทางที่ไม่จำเป็นต้องถูกเรียกใช้ในทุก Query เช่น ขนาดน้ำหนัก ประเทศผลิต การแยกตารางช่วยลดขนาด Table และประหยัดหน่วยความจำ	ข้อมูลที่เป็น Child Entities หรือรายการที่เติบโตได้เรื่อยๆ เช่น ความคิดเห็น รายการคำสั่งซื้อย่อย ประวัติการทำธุรกรรม

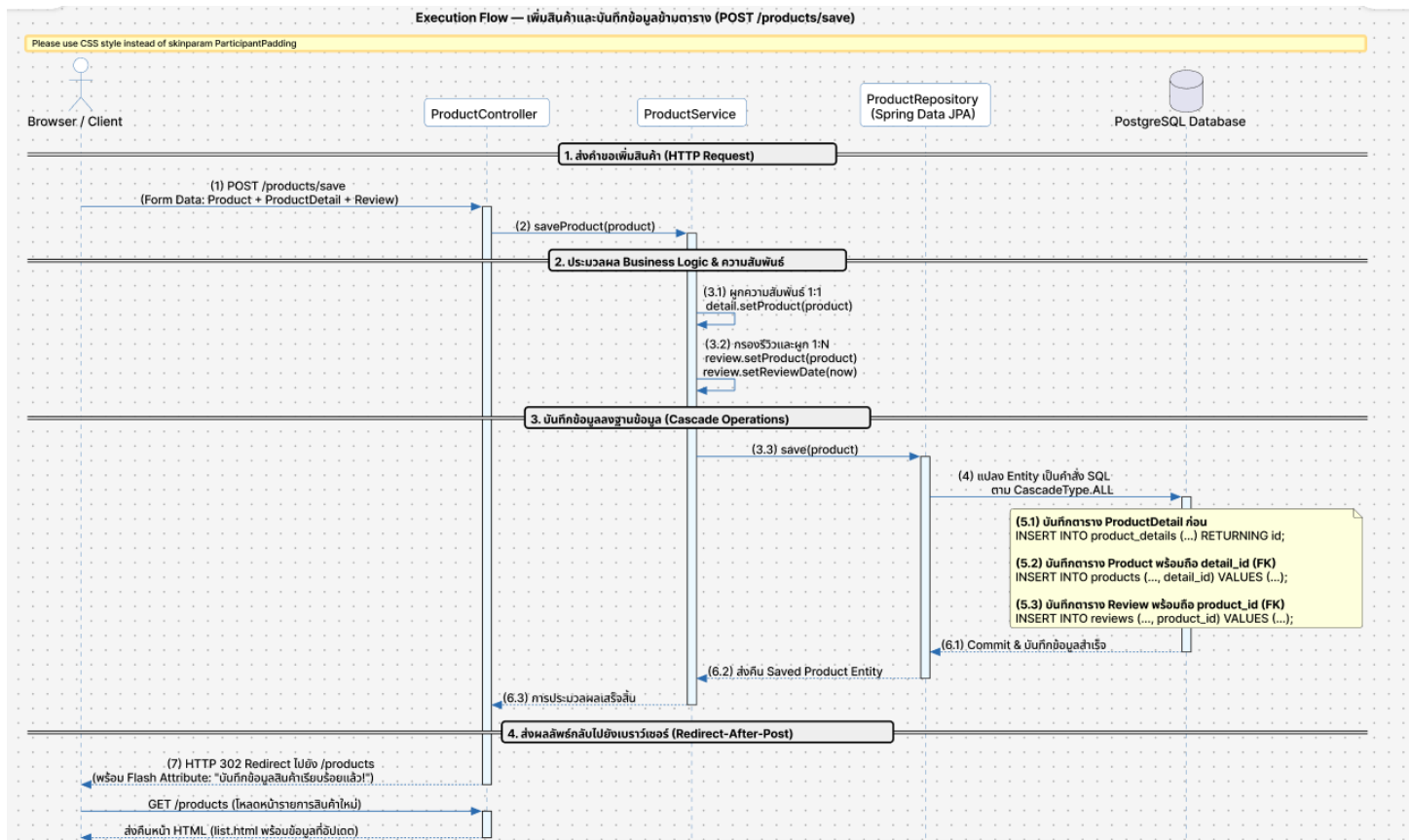
1.3 การออกแบบ Strategy Pattern สำหรับคำนวณส่วนลด

การคำนวณราคาสินค้าที่มีโปรโมชั่นต่างกัน เช่น ไม่มีส่วนลด สมาชิก 10% เทศกาล 20% ถูกออกแบบด้วย Strategy Pattern เพื่อแยกอัลกอริทึมการคำนวณออกจากคลาสที่เรียกใช้งาน



1.4 Execution Flow (เส้นทางการทำงานตั้งแต่ HTTP Request → DB → View)

เมื่อผู้ใช้ส่งคำขอเพิ่มสินค้าใหม่เข้ามาในระบบ ขั้นตอนการไหลของข้อมูลจะเป็นไปตามสถาปัตยกรรมแบบแบ่งชั้น (Layered Architecture) ดังนี้



Flow การทำงานดังนี้

- Request (ข้อ 1 - 2): ผู้ใช้กด Submit ฟอรม์จากหน้าเว็บ Browser จะส่งข้อมูลผ่าน HTTP POST เข้ามาที่ Controller ซึ่งจะส่งต่อ Object สินค้าไปให้ Service ทำงาน
- Business Logic (ข้อ 3.1 - 3.2): Service ทำหน้าที่ตรวจสอบและผูก Foreign Key ของ Object ทั้งสองฝั่ง (Bidirectional) ให้ชี้หากันอย่างสมบูรณ์
- Database Persistence (ข้อ 3.3 - 6): ด้วย CascadeType.ALL ของ JPA ตัว Repository จะสั่ง SQL INSERT เรียงตามลำดับความสัมพันธ์:
 - สร้าง product_details ก่อนเพื่อเอา Primary Key
 - สร้าง products พร้อมใส่ Foreign Key detail_id
 - สร้าง reviews พร้อมใส่ Foreign Key product_id
- Response (ข้อ 7): Controller ใช้รูปแบบ Post/Redirect/Get (PRG Pattern) ส่งคำสั่ง Redirect กลับไปที่หน้ารายการสินค้าพร้อมข้อความแจ้งเตือน เพื่อป้องกันการ Submit ฟอรม์ซ้ำเมื่อกด Refresh หน้าจอ

ภาคที่ 2: โค้ดโปรแกรมและคำอธิบาย (Code & Technical Explanation)

2.1 Entity ทั้ง 3 ตัว และคำอธิบาย Annotations

การออกแบบ Entity ในระบบใช้ Jakarta Persistence API (JPA) ในการจับคู่ (Mapping) ระหว่าง Class ของภาษา Java เข้ากับตารางจริงในฐานข้อมูล PostgreSQL

1) Product.java (Entity หลัก — Owner ของความสัมพันธ์ 1:1 และ 1:N)

คลาสนี้เป็นศูนย์กลางของข้อมูลสินค้า ทำหน้าที่เป็นเจ้าของความสัมพันธ์ (Owner Side) ของทั้ง ProductDetail และ Review

```
@Entity
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private Double price;
    private String discountType = "NONE";

    // — 1:1 กับ ProductDetail (ฝั่ง Owner ถือ FK) —
    @OneToOne(cascade = CascadeType.ALL, orphanRemoval = true)
    @JoinColumn(name = "detail_id", referencedColumnName = "id")
    private ProductDetail detail = new ProductDetail();

    // — 1:N กับ Review (มีได้หลายรีวิว) —
    @OneToMany(mappedBy = "product", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Review> reviews = new ArrayList<>();

    // — ราคาสุทธิคำนวณสด ไม่บันทึกลงฐานข้อมูล —
    @Transient
    private Double discountedPrice;
}
```

Annotations สำคัญ:

- `@Entity`: ระบุว่าคลาสนี้เป็น JPA Entity ที่จะถูกแปลงเป็นตารางในฐานข้อมูล
- `@Table(name = "products")`: กำหนดชื่อตารางใน PostgreSQL ให้ชื่อว่า products
- `@Id`: กำหนดให้ตัวแปร id ทำหน้าที่เป็น Primary Key (PK) ประจำตาราง
- `@GeneratedValue(strategy = GenerationType.IDENTITY)`: ให้ฐานข้อมูลสร้างค่า PK ให้อัตโนมัติแบบ Auto-increment (ตรงกับชนิด BIGSERIAL ใน PostgreSQL)
- `@OneToOne(...)`: ประกาศความสัมพันธ์แบบ 1:1 โดยมีคุณสมบัติเพิ่มเติม:
 - `cascade = CascadeType.ALL`: ถ่ายทอดคำสั่งทุกอย่าง (บันทึก, แก้ไข, ลบ) ไปยัง ProductDetail ด้วยอัตโนมัติ
 - `orphanRemoval = true`: หากตัดการเชื่อมโยงความสัมพันธ์ ข้อมูลลูกที่เค้างค์จะถูกลบออกจากฐานข้อมูลทันที
- `@JoinColumn(name = "detail_id", referencedColumnName = "id")`: ประกาศว่าตาราง products เป็นผู้ถือ Foreign Key ชื่อคอลัมน์ detail_id ซึ่งชี้ไปยัง PK (id) ของตาราง product_details
- `@OneToMany(mappedBy = "product", ...)`: ประกาศความสัมพันธ์แบบ 1:N โดยระบุ mappedBy = "product" เพื่อบอก JPA ว่าฝั่ง Product ไม่ได้ถือ FK เอง แต่ความสัมพันธ์นี้ถูกควบคุมโดยฟิลด์ product ในคลาส Review
- `@Transient`: ระบุว่าฟิลด์ discountedPrice (ราคาสุทธิ) ใช้สำหรับการคำนวณและแสดงผลบนหน้าเว็บเท่านั้น ไม่ต้องสร้างเป็นคอลัมน์ลงในฐานข้อมูล

2) ProductDetail.java (ตารางข้อมูลเสริมสินค้า — ฝั่ง Inverse ของ 1:1)

ทำหน้าที่เก็บข้อมูลเชิงเทคนิคหรือรายละเอียดเพิ่มเติมของสินค้า ออกแบบแยกออกมาตามหลัก SRP (Single Responsibility Principle) เพื่อให้ตารางหลักมีขนาดใหญ่เกินความจำเป็น

```
@Entity
@Table(name = "product_details")
public class ProductDetail {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(columnDefinition = "TEXT")
    private String description;

    private String warranty;
    private Double weight;
    private String dimensions;
    private String manufacturedCountry;

    // ———> ซ้ำกลับไปยัง Product (Inverse Side) ———>
    @OneToOne(mappedBy = "detail")
    private Product product;
}
```

Annotations สำคัญ:

- `@Column(columnDefinition = "TEXT")`: สั่งให้สร้างคอลัมน์ description เป็นชนิด TEXT ใน PostgreSQL เพื่อให้สามารถเก็บข้อความบรรยายคุณสมบัติสินค้าที่มีความยาวมากๆ ได้โดยไม่จำกัดความยาวที่ 255 ตัวอักษร
- `@OneToOne(mappedBy = "detail")`: ประกาศตัวเป็น Inverse Side (ฝั่งรอง) ของความสัมพันธ์ 1:1 เชื่อมโยงกลับไปยังฟิลด์ชื่อ detail ในคลาส Product ทำให้สามารถเรียกดูข้อมูลสินค้าเจ้าของย้อนกลับได้แบบสองทาง (Bidirectional)

3) Review.java (ตารางรีวิวสินค้า — ฝั่ง Many ของ 1:N ที่ถือครอง Foreign Key)

เก็บข้อมูลความคิดเห็นและคะแนนดาวของผู้ใช้งาน ออกแบบให้แยกเป็นอิสระเพื่อรองรับการเติบโตของข้อมูลตามหลัก OCP (Open/Closed Principle)

```
@Entity
@Table(name = "reviews")
public class Review {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String reviewer;
    private Integer rating;

    @Column(columnDefinition = "TEXT")
    private String comment;

    private LocalDate reviewDate;

    // — ฝั่ง Many เป็นผู้ถือ Foreign Key เสมอ —
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "product_id")
    private Product product;
}
```

Annotations สำคัญ:

- `@ManyToOne(fetch = FetchType.LAZY)`: กำหนดความสัมพันธ์แบบ N:1 (หลายรีวิวผูกกับสินค้า 1 ชิ้น) พร้อมกำหนดกลยุทธ์การดึงข้อมูลเป็นแบบ LAZY เพื่อไม่ให้โหลดข้อมูล Product มาโดยไม่จำเป็นจนกว่าจะมีการเรียกใช้งานจริง ช่วยประหยัดทรัพยากรระบบ
- `@JoinColumn(name = "product_id")`: กำหนดให้สร้างคอลัมน์ Foreign Key ชื่อ `product_id` ในตาราง `reviews` เพื่อเชื่อมโยงไปยัง `id` ของสินค้า

2.2 โค้ด Service, Controller และการอธิบาย Constructor Injection

1) ProductService.java (Business Logic Layer)

ทำหน้าที่ประมวลผลตรรกะทางธุรกิจ ควบคุมการคำนวณส่วนลด และจัดการความสัมพันธ์ของข้อมูลก่อนส่งต่อให้ Repository

```
@Service
public class ProductService {

    private final ProductRepository productRepository;
    private final DiscountContext discountContext;

    // —— Constructor Injection ——
    public ProductService(ProductRepository productRepository, DiscountContext discountContext) {
        this.productRepository = productRepository;
        this.discountContext = discountContext;
    }

    public List<Product> getAllProducts() {
        List<Product> products = productRepository.findAll();

        // นำราคาสินค้ามาคำนวณราคาสุทธิผ่าน Strategy Pattern
        for (Product product : products) {
            discountContext.setStrategy(discountContext.getStrategyByType(product.getDiscountType()));
            double price = (product.getPrice() != null) ? product.getPrice() : 0.0;
            product.setDiscountedPrice(discountContext.calculate(price));
        }
        return products;
    }

    public void saveProduct(Product product) {
        // ผูกความสัมพันธ์สองทางให้ครบถ้วนก่อนบันทึก
        if (product.getDetail() != null) {
            product.getDetail().setProduct(product);
        }
        productRepository.save(product);
    }
}
```

2) ProductController.java (Presentation Layer)

ทำหน้าที่เป็นตัวกลางรับส่ง HTTP Request/Response โดยไม่บรรจุ Business Logic หรือคำสั่งเรียกใช้งานฐานข้อมูลโดยตรง

```
@Controller
public class ProductController {

    private final ProductService productService;

    // —— Constructor Injection ——
    public ProductController(ProductService productService) {
        this.productService = productService;
    }

    @GetMapping("/{products}", "/products/")
    public String listProducts(Model model) {
        model.addAttribute("products", productService.getAllProducts());
        return "products/list";
    }

    @PostMapping("/products/save")
    public String saveProduct(@ModelAttribute("product") Product product, RedirectAttributes
redirectAttributes) {
        productService.saveProduct(product);
        redirectAttributes.addFlashAttribute("message", "บันทึกข้อมูลสินค้าเรียบร้อยแล้ว!");
        return "redirect:/products";
    }
}
```

2.3 ทำไมต้องใช้ Constructor Injection?

ในโค้ดทั้ง ProductController และ ProductService มีการเชื่อมต่อ Dependencies โดยใช้ Constructor Injection (การฉีด Dependency ผ่านคอนสตรักเตอร์) แทนการใช้ @Autowired และไวน์บฟิลด์โดยตรง (Field Injection) ด้วยเหตุผลสำคัญ 4 ประการ:

1. สอดคล้องกับหลัก Dependency Inversion Principle (DIP) และ IoC:

คลาสระดับสูงไม่จำเป็นต้องรู้ว่า Dependencies ถูกสร้างขึ้นอย่างไร หรือต้องใช้คำสั่ง new ด้วยตัวเอง คลาสเพียงแค่ประกาศความต้องการ (Requirements) ผ่านพารามิเตอร์ของ Constructor แล้วปล่อยให้ Spring IoC Container เป็นผู้จัดส่ง Instance ที่เหมาะสมเข้ามาให้

2. รองรับ Immutability (ความคงที่ของข้อมูล):

ช่วยให้เราสามารถประกาศฟิลด์เป็น private final ได้ ซึ่งเป็นการรับประกันว่า Dependency นั้นจะถูกกำหนดค่าทันทีตั้งแต่ตอนเริ่มสร้าง Object และจะไม่สามารถถูกแก้ไข หรือถูกเขียนทับได้อีกตลอดอายุการทำงาน ป้องกันปัญหา State เปลี่ยนแปลงโดยไม่ตั้งใจ

3. ป้องกันข้อผิดพลาด NullPointerException (Safe Initialization):

หากระบบขาด Dependency ตัวใดตัวหนึ่ง หรือคอนฟิกรูมผิดพลาด Object นั้นจะไม่สามารถถูกสร้างขึ้นได้เลย (Compile/Startup Fail ทันที) ทำให้เรารู้จุดบกพร่องของระบบตั้งแต่ตอนเริ่มรันเซิร์ฟเวอร์

4. สะดวกต่อการทำ Unit Testing (High Testability):

ในการเขียนชุดทดสอบ (Unit Test) นักพัฒนาสามารถสร้าง Instance ของ Service หรือ Controller ขึ้นมาทดสอบได้ทันที โดยส่ง Mock Object (เช่น Mockito Mock) ผ่าน Constructor เข้าไปได้โดยตรง โดยไม่จำเป็นต้องบูต Spring ApplicationContext ทั้งหมดขึ้นมาให้สิ้นเปลืองเวลา

ภาคที่ 3: ภาพหน้าจอ ชื่อ นสามสกุล รหัสนักศึกษา

3.1 ในเว็บ localhost

Product Shop

Lab 8 — 1:1 & 1:N Relationship

+ เพิ่มสินค้าใหม่

#	ชื่อสินค้า	หมวดหมู่	ยี่ห้อ	คลัง	ราคา (บาท)	ส่วนลด	ราคาสุทธิ	รับประกัน (1:1)	น้ำหนัก (1:1)	รีวิว (1:N)	จัดการ
1	iPhone 15 Pro Max	Electronic	Apple	500	17500.00	MEMBER	15750.00			0 รีวิว	แก้ไขลบ
2	673380428-6	Student	Sitthichok Moknak	1	9999999999.00	NONE	9999999999.00			1 รีวิว	แก้ไขลบ
4	gg	gg	gg	500	500.00	SEASONAL	400.00	1 Year	50.0	1 รีวิว	แก้ไขลบ
5	สินค้าทดสอบ	Test	Tester	100	999.00	MEMBER	899.10	รับประกันสูงสุด สลายตา	1.0	1 รีวิว	แก้ไขลบ

3.2 ใน postgresSQL pgadmin4

Publications

Schemas (1)

public

Aggregates

Collations

Domains

FTS Configurations

FTS Dictionaries

FTS Parsers

FTS Templates

Foreign Tables

Functions

Materialized Views

Operators

Procedures

Sequences

Tables (3)

product_details

products

reviews

Trigger Functions

Types

Views

1

2

SELECT * FROM public.product_details

ORDER BY id ASC

Data Output

Messages

Notifications

Showing rows: 1 to 4

Page No:

	id [PK] bigint	description text	dimensions character varying (255)	manufactured_country character varying (255)	warranty character varying (255)	weight double precision
1	1					[null]
2	2					[null]
3	4	gg		China	1 Year	50
4	5	สำหรับทดสอบ	100*100*100	ประเทศทดสอบ	รับประกันสูงสุดสลายตา	1

Languages

Publications

Schemas (1)

public

- Aggregates
- Collations
- Domains
- FTS Configurations
- FTS Dictionaries
- FTS Parsers
- FTS Templates
- Foreign Tables
- Functions
- Materialized Views
- Operators
- Procedures
- Sequences
- Tables (3)
 - product_details
 - products
 - reviews
- Trigger Functions
- Types
- Views

Subscriptions

postgres

login/Group Roles

Query Query History

1 SELECT * FROM public.products

2 ORDER BY id ASC

Scratch Pad

Data Output Messages Notifications

Showing rows: 1 to 4 of 1

	id [PK] bigint	brand character varying (255)	category character varying (255)	discount_type character varying (255)	name character varying (255)	price double precision	stock integer	detail_id bigint
1	1	Apple	Electronic	MEMBER	iPhone 15 Pro Max	17500	500	1
2	2	Sittichok Moknak	Student	NONE	673380428-6	99999999999	1	2
3	4	gg	gg	SEASONAL	gg	500	500	4
4	5	Tester	Test	MEMBER	สินค้าทดสอบ	999	100	5

Successfully run. Total query runtime: 117 msec. 4 rows affected.

PostgreSQL Local/lab8shop - Database connected

Publications

Schemas (1)

public

- Aggregates
- Collations
- Domains
- FTS Configurations
- FTS Dictionaries
- FTS Parsers
- FTS Templates
- Foreign Tables
- Functions
- Materialized Views
- Operators
- Procedures
- Sequences
- Tables (3)
 - product_details
 - products
 - reviews
- Trigger Functions
- Types
- Views

Query Query History

1 SELECT * FROM public.reviews

2 ORDER BY id ASC

Scratch Pad

Data Output Messages Notifications

Showing rows: 1 to 3 of 1

	id [PK] bigint	comment text	rating integer	review_date date	reviewer character varying (255)	product_id bigint
1	1	AA	5	2026-09-10	A	2
2	2	gg	5	2026-09-10	A	4
3	3	รีวิวทดสอบ	4	2026-09-11	นายทดสอบ สมมติดี	5

Successfully run. Total query runtime: 108 msec. 3 rows affected.